Debugging Flaky Tests using Spectrum-based Fault Localization

Martin Gruber
BMW Group, University of Passau
Munich, Germany
martin.gr.gruber@bmw.de

Gordon Fraser
University of Passau
Passau, Germany
gordon.fraser@uni-passau.de

arXiv:2305.04735v1 [cs.SE] 8 May 2023

***Abstract*—Non-deterministically behaving (i.e., *flaky*) tests hamper regression testing as they destroy trust and waste computational and human resources. Eradicating flakiness in test suites is therefore an important goal, but automated debugging tools are needed to support developers when trying to understand the causes of flakiness. A popular example for an automated approach to support regular debugging is spectrum-based fault localization (SFL), a technique that identifies software components that are most likely the causes of failures. While it is possible to also apply SFL for locating likely sources of flakiness in code, unfortunately the flakiness makes SFL both imprecise and non-deterministic. In this paper we introduce SFFL (Spectrum-based Flaky Fault Localization), an extension of traditional coverage-based SFL that exploits our observation that 80% of flaky tests exhibit varying coverage behavior between different runs. By distinguishing between stable and flaky coverage, SFFL is able to locate the sources of flakiness more precisely and keeps the localization itself deterministic. An evaluation on 101 flaky tests taken from 48 open-source Python projects demonstrates that SFFL is effective: Of five prominent SFL formulas, DStar, Ochiai, and Op2 yield the best overall performance. On average, they are able to narrow down the fault's location to 3.5% of the project's code base, which is 18.7% better than traditional SFL (for DStar). SFFL's effectiveness, however, depends on the root causes of flakiness: The source of non-order-dependent flaky tests can be located far more precisely than order-dependent faults.**

***Index Terms*—Flaky Tests; Fault Localization; Debugging aids**

I. Introduction

Flaky tests are tests that behave non-deterministically, i.e., they can both pass and fail on the same software version. Test flakiness negatively impacts software development, especially continuous integration [1], [2]: Before merging their changes, developers want to test them to either find potential bugs early or merge with the confidence that they did not break existing functionality. The assumption here is that when all tests passed before the change, and one or more tests fail afterwards, these failures are caused by the change and indicate a regression. For flaky tests, however, this assumption is not true since they may fail intermittently, independently of a change. Since distinguishing flaky from non-flaky failures is a non-trivial task [3]–[12], developers need to investigate flaky failures and inspect the relevant code, mostly only to find that the failure had nothing to do with their changes.

Recent developer surveys confirmed this observation, finding that flakiness wastes developer time and makes it harder to merge pull requests [1] and that developers who experience flakiness more often become frustrated and are more likely to ignore genuine test failures [2]. The latter is particularly problematic since—while most flaky tests are indeed false alarms [13], [14]—test flakiness can also be caused by sporadic bugs in the system under test, which might lead to production failures. Flaky tests are therefore a pressing issue that needs to be addressed, and ignoring them might lead to severe consequences on developer trust and product quality.

By asking developers what information they require for fixing a flaky test, recent research found that the code elements involved in the flakiness are valuable pieces of information, which, however, are difficult to obtain in complex systems [14], and that developers mainly wish for automated debugging tools to better deal with test flakiness [1]. We follow this call by adapting spectrum-based fault localization (SFL), a popular and established automated debugging technique, specifically for debugging flaky tests, aiming to identify the exact source code line that causes a test to behave flaky (i.e., the flaky fault). Unlike traditional SFL, SFFL (Spectrum-based Flaky Fault Localization) takes into account that tests may show different coverage behaviors, which we observe for 80.3 % of flaky tests in our evaluation: For tests that are known to be flaky (e.g., through reruns or other flakiness detection techniques), SFFL only considers a statement as covered, if all test runs covered it. This follows the intuition that the fault must be located before the failure appears, i.e., before the coverage behavior diverges. For non-flaky tests on the other hand, it considers a statement as covered if it was involved in any test execution. The main contributions of this paper are as follows:

- **Approach:** We propose SFFL, a variation of SFL that is specifically designed for debugging flaky tests by considering all observed coverage behaviors of all tests. Our method both increases the effectiveness of fault localization and keeps the fault localization itself mostly deterministic.
- **Implementation:** We integrate SFFL in FlaPy [15], a tool for mining flaky tests in a given set of Python projects.
- **Dataset:** We provide a dataset of manually labeled fault locations for 227 flaky tests.
- **Evaluation:** We assess the fault identifying performance of SFFL on our dataset using five established SFL formulas: Tarantula, Ochiai, DStar, Op2, and Barinel.

In our evaluation, the best performing formulas (Ochiai,

DStar, and Op2) were able to achieve a mean average-case EXAM score of 0.035. That is, a flaky fault could on average be located after inspecting $3.5\,\%$ of the project's code base. Compared to the traditional form of SFL that only considers a single coverage behavior for each test, this is a significant improvement of $18.7\,\%$, making SFFL practically applicable: The faults of $55.6\,\%$ of all flaky tests could be identified by inspecting the top ten ranked statements.

While investigating the fault localization performance on flaky tests of different root causes, we found SFFL to perform notably better on non-order-dependent (NOD) flaky tests than on order-dependent (OD) ones. This is expected, since OD flakiness can be interpreted as being located either inside other test cases or in the absence of adequate state-resetting code, and both cases are hard to identify for any SFL approach. There are already specialized approaches for debugging order-dependencies [16]–[19], which however do not apply to NOD flaky tests, such that SFFL provides a great addition and contribution to the overall flakiness debugging toolkit available to developers and researchers. SFFL can serve as a foundation for further approaches targeting flakiness, such as flakiness detection, program behavior visualization, automated root causing, or automated fixing. We make all implementations, data, and evaluation scripts available openly [20] and hope that they will foster future research on debugging flaky tests.

II. Background

Fault localization is the process of identifying the component within a system that is responsible for a given (test) failure (i.e., the fault). Automated fault localization techniques [21] aim to reduce the effort of manual debugging, but also provide a foundation for other automated methods, such as program repair [22]. Among these, spectrum-based fault localization approaches (SFL) form one of the largest and most popular sub-groups: Their main idea is to assign each component a suspiciousness score based on the passing and failing executions it was involved in (i.e., the coverage behavior of the tests). The more failing and the fewer passing executions in which the component participated, the more suspicious it is, i.e., the more likely it is to contain the fault that causes the observed failures. When sorting the components by their suspiciousness scores, a developer, tester, or automated technique receives a ranked list of components in which the fault is most likely located. SFL can be applied at various granularities, components can for example be statements, classes, modules, or environment factors (e.g., operating system or no. of CPU cores).

Previous research proposed multiple concrete formulas for calculating the suspiciousness score. Some popular ones are depicted in Table I. The term s represents a component (in most cases a statement), and $S(s)$ denotes its suspiciousness score. $passed(s)$ and $failed(s)$ are the number of passed/failed executions where s was involved/covered. $totalpassed$ and $totalfailed$ denote the total number of passed and failed executions. Note that the value range of Tarantula, Ochiai, and Barinel is $[0, 1]$, whereas the value range of DStar is $[0, \infty[$ and the value range of Op2 is $[-\frac{totalpassed}{totalpassed+1}, totalfailed]$.

Table I: SFL formulas [23]. For DStar we set the exponent * to 2, as recommended by Wong et al. [24].

Tarantula [25]:	$S(s) = \frac{failed(s)/totalfailed}{failed(s)/totalfailed+passed(s)/totalpassed}$
Ochiai [26]:	$S(s) = \frac{failed(s)}{\sqrt{totalfailed \cdot (failed(s)+passed(s))}}$
DStar [24]:	$S(s) = \frac{failed(s)^*}{passed(s)+(totalfailed-failed(s))}$
Op2 [27]:	$S(s) = failed(s) - \frac{passed(s)}{totalpassed+1}$
Barinel [28]:	$S(s) = 1 - \frac{passed(s)}{passed(s)+failed(s)}$

Over the past decade, SFL has been applied and evaluated in multiple contexts on both artificial and real faults [23]. However, the traditional SFL approach assumes the tests to behave deterministically, i.e., to always produce the same test outcome given the same version of the system under test. This is not the case for flaky tests, which pass and fail intermittently, for example due to networking issues, randomness, or concurrency. Recent studies have investigated the impact of flaky tests on automated debugging techniques [29]–[31]. It was found that flakiness corrupts spectrum-based fault localization, resulting in a call for novel fault localization algorithms that take test flakiness into account. We follow that call by extending SFL for debugging flaky tests, making it more reliable and effective.

To the best of our knowledge, so far there have been two other attempts at applying SFL for debugging flaky tests rather than regular test failures: Habchi et al. [32] proposed an SFL approach to identify the classes that are responsible for non-deterministic test behavior. Besides adjusting the SFL formulas for locating the faults of flaky instead of persistently failing tests, they augmented the fault localization with change and code metrics. FlakyLoc [33] uses a spectrum-based approach together with combinatorial testing to identify the environmental factors (network bandwidth, browser, screen resolution, ...) that cause tests in web applications to behave flaky. However, neither of these approaches aims at identifying the exact statement in the source code that is responsible for flaky test behavior. Furthermore, they do not utilize the potentially different coverage behaviors of flaky tests, which we identify as a core issue but also an opportunity. Our approach (SFFL) addresses both aspects.

The Flakiness Debugger proposed by Ziftci and Cavalcanti [34] is another technique aiming at debugging flaky tests. Like SFFL, it identifies statements related to flaky behavior. Unlike our approach, however, it is not based on SFL, but on a custom `DIVERGENCE` algorithm that identifies the first point of divergence in the control flow of each failing run from any of the passing runs. Therefore, it requires a more complex instrumentation that records the entire dynamic execution trace, for which they use Google-internal technology. SFFL on the other hand relies only on code coverage information, which can be collected using openly available tools [35], [36].

Table II: Fault localization matrix of an example test suite illustrating how SFFL works compared to SFL. Rows correspond to source code lines. *totalstable* and *totalflaky* have value 1. This code example is similar to the one used by Ziftci et al. [34].

*=fault		run1 t1 pass	run1 t2 pass	run2 t1 fail	run2 t2 pass	$\cap t1$ (flaky)	$\cup t2$ (stable)	SFL Tarantula run1	SFL Tarantula run2	SFFL Tarantula $\cap t1 + \cup t2$
1	`def gen_int():`	✓	✓	✓	✓	✓	✓	0.5	0.5	0.5
*2	`    r_int = random.randint(0, 100)`	✓	✓	✓	✓	✓	✓	0.5	0.5	0.5
*3	`    if r_int >= 50:`	✓	✓	✓	✓	✓	✓	0.5	0.5	0.5
4	`        return "Big number"`			✓	✓		✓	0.0	0.5	0.0
5	`    else:`						✓	0.0	0.0	0.0
6	`        return "Small number"`	✓	✓				✓	0.5	0.0	0.0
7										
8	`#flaky test`									
9	`def t1():`									
10	`    assert "Small" in gen_int()`									
11										
12	`#stable test`									
13	`def t2():`									
14	`    assert "number" in gen_int()`									

III. Applying Spectrum-based Fault Localization to Flaky Tests

A. Approach

Unlike deterministically failing tests, flaky tests often have at least two different coverage behaviors: one for which they pass and one for which they fail. For example, consider Table II, which shows a test suite containing two tests, one of them flaky (`t1`), the other one stable (`t2`). Both tests execute the function `gen_int` and make assertions on its return value. The function itself generates a random integer (line 2) and returns different strings depending on its value (line 3–6). In this example, the fault that causes `t1` to behave flaky is located in lines 2 and 3, because this is where the failure-causing non-deterministic value is generated (line 2) and where it is propagated towards the return value and ultimately the assertion (line 3). In contrast, the assertion of `t2` passes independently of which string is returned. The test is therefore not flaky.

Arguably, one could also see the fault to be located in line 10, where the test makes an assertion on a non-deterministic value. However, our goal is to find the location where the failure-causing non-determinism first occurs. Whether this non-determinism should be eradicated, or if the assertion needs to be relaxed, depends on the specification of the system (oracle problem) and can therefore not be universally decided. Knowing the source and location of the non-determinism causing flaky failures should always be helpful when debugging flaky tests, regardless of where the fix is applied.

When executing the tests multiple times, we can observe two different coverage behaviors (run1 and run2), which happen to be the same for both tests for this simple example. In our experiments (Section IV-A), we found that $80.3\,\%$ of all flaky tests had at least two different coverage behaviors (i.e., there were statements that they did not always cover). On the other hand, this was only true for $11.9\,\%$ of all stable (non-flaky) tests. If we perform traditional SFL, for example using Tarantula, lines 1, 2, 3, as well as line 4 (run1) or line 6 (run2) receive the highest suspiciousness scores. Not only did the fault localization wrongly mark lines 1, 4, and/or 6, the fault localization itself became non-deterministic, making it less usable and potentially subject to the same loss of trust as tests that become flaky.

Table III: SFFL formulas (also used by Habchi et al. [32])

Tarantula [25]:	$S(s) = \frac{flaky(s)/totalflaky}{flaky(s)/totalflaky + stable(s)/totalstable}$
Ochiai [26]:	$S(s) = \frac{flaky(s)}{\sqrt{totalflaky \cdot (flaky(s) + stable(s))}}$
DStar [24]:	$S(s) = \frac{flaky(s)^*}{stable(s) + (totalflaky - flaky(s))}$
Op2 [27]:	$S(s) = flaky(s) - \frac{stable(s)}{totalstable+1}$
Barinel [28]:	$S(s) = 1 - \frac{stable(s)}{stable(s)+flaky(s)}$

The main idea of SFFL is to capitalize on the multi-coverage behavior of flaky tests to narrow down the set of potential fault locations, making it both more effective than SFL and deterministic: The statement causing the test to behave flaky (the fault) needs to be covered in every test execution, regardless whether the test passes or fails. A statement that is covered only in passing or only in failing runs can only be the consequence of a flaky fault, but cannot contain the fault itself. In other words, the faulty non-deterministic value must be created before the coverage behavior diverges. The fault must therefore be located in the intersection of all coverage behaviors (i.e., covered by both passing and failing executions). We consider a statement to be covered by a flaky test iff *all* its executions covered it ($\cap t1$). For stable tests we apply the same logic in opposite direction: The faulty statement has the potential of making any test flaky that covers it. A statement that is covered by a stable test is therefore less likely to be faulty. When a stable test exhibits different coverage behaviors, we take their union, i.e., we consider a statement as covered by a stable test iff *any* of its executions covered it ($\cup t2$).

In detail, SFFL works as follows:

1) Create the fault localization matrix based on all runs

including repetitions.
2) Create an error vector in the fault localization matrix based on flakiness ($failed \rightarrow \mathit{flaky}$, $passed \rightarrow stable$).
3) Add columns for coverage intersection for flaky tests and union for stable ones.
4) Calculate $totalflaky$ and $totalstable$.
5) For each statement s determine $stable(s)$, $\mathit{flaky}(s)$, and calculate its suspiciousness score (Table III).
6) Rank statements based on suspiciousness scores.

By applying SFFL to our example (last column in Table II), we can prevent the fault localization from becoming flaky and improve its effectiveness by eliminating lines 4 and 6 as potential fault locations.

B. Limitations

1) Multiple faults: In our concrete implementation we base SFFL on single fault localization techniques, meaning that our approach assumes one fault per test suite and that its performance might decrease when multiple faults are present. In our evaluation sample, 12 out of 48 projects contributed more than one flaky test. From these, 8 had the same fault location for all its flaky tests. This shows that the single fault assumption is not unreasonable, but multiple faults are also present in flaky test suites. In principle, SFFL could also integrate existing work on multiple fault localization (e.g., [28]).

2) Multiple executions needed: To apply SFFL, multiple test executions are needed that exhibit both passing and failing test outcomes of the flaky tests. While this can make a cold start search for flaky faults computationally challenging (as the search for flaky tests itself), it should not be a concern for the practical usage of our approach when debugging an individual flaky test: The question of where a fault is located usually arises *after* a failure occurred. When collecting coverage information in every test execution, the failing coverage behavior is therefore already present and all that is needed is the test's coverage behavior in the passing case. This should be easily retrievable, since (1) flaky tests are mostly rerun anyway [1], [2], [37], and (2) most flaky tests have low failure rates [15], so they are likely to produce a passing execution within a few reruns. If, however, our test suite contains multiple flaky tests that share the same fault, we need the different coverage behaviors of not one but all flaky tests in the test suite to achieve optimal performance, which might require a substantial number of test executions. A possible way to achieve this amount of reruns could be by applying DeFlaker [3], a tool that identifies regression test executions that were effectively conducted on the same software version (from the perspective of a certain test case) and can therefore be seen as reruns.

3) Coverage needed: Like the traditional SFL techniques we build on, our approach relies on coverage information, which might be hard to collect in certain scenarios, like the embedded system domain [38]. Furthermore, the instrumentation needed to retrieve coverage information might also change the behavior of the program and affect the flakiness. Nevertheless, the concept we present can also be applied to other components (such as lines in log files), or on a more sparse granularity.

4) Flakiness in test code: SFFL determines the suspiciousness of a statement by the number of flaky and stable tests that execute it. This makes SFFL less suitable for debugging tests whose flakiness resides in the test code instead of the code under test, which is particularly common for order-dependent flaky tests.

C. Implementation

To implement SFFL, we extend FlaPy [15], a tool that allows researchers and developers to identify flaky tests in a given set of Python projects by repeatedly executing their test suites. We chose this option as FlaPy takes care of the isolation of individual test runs, contributing to accurate and realistic results, as well as dependency installation and the parallelization of the experiments, allowing us to conduct our evaluation on a large set of projects. Furthermore, it provides a convenient interface for collecting and analyzing the test outcomes.

FlaPy relies on pytest [39] to execute the projects' test suites and already uses the pytest-cov plugin [36] to generate XML coverage reports, which however do not distinguish between the coverage of individual test cases. We extend FlaPy by using pytest-cov's `--cov-context=test` option [40] to record the individual coverage of each test case, which is stored in an sqlite database. We also extend FlaPy's results-parser to accumulate the coverage information from the different test executions and perform the fault localization. For order-dependent flaky tests we only consider test executions conducted in random order, and for non-order-dependent flaky tests we only consider those conducted in the same (i.e., default) order.

When implementing the SFL formulas, one has to take division by zero issues into account. These can occur when the total number of passed executions is zero (Tarantula) or when a statement was covered by all failing and by no passing execution (DStar). We mitigated division by zero issues by using a save division function that returns zero for 0/0 (an appropriate assumption in our specific case) and infinity for x/0 where $x > 0$ (negative x values do not occur in the formulas).

IV. Evaluation

To assess the efficacy of SFFL, we measure both its effectiveness, and the improvement it yields over traditional SFL. Namely, we address the following research questions:

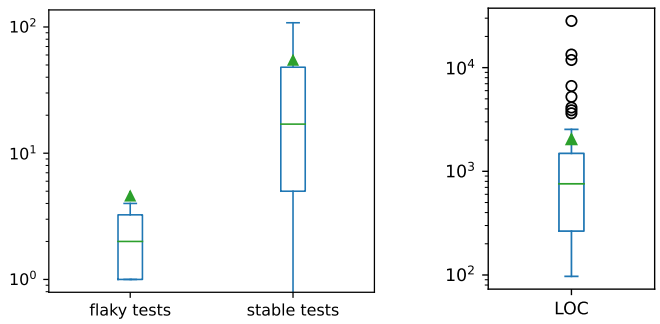
RQ1 (Effectiveness): How well can SFFL detect non-order-dependent flaky faults?

RQ2 (Improvement): How much better can SFFL locate non-order-dependent flaky faults compared to traditional SFL?

RQ3 (Root Cause Differences): How well does SFFL perform on flaky tests of different root causes?

A. Experimental Setup

To evaluate the fault locating performance of SFFL, we utilize a dataset of flaky tests in Python projects we created in prior work [15], which contains 7 571 flaky tests from 1 006 projects. However, as flakiness is notoriously difficult to reproduce we did not rely on our old findings about which tests are flaky. Instead, we re-executed the projects' test suites



(a) Number of flaky and stable tests per project

(b) Lines of source code per project

Figure 1: Statistics about the 48 projects in our dataset. Note the logarithmic y-axes. (triangles=means)

400 times in same order and 400 times in random order, which is twice as often as the original study [15]. This large number of reruns provides more confidence in the correct classification of a test as flaky or non flaky, which increases the validity of our evaluation. We conducted the test executions using our extended version of FlaPy, which collects the individual line coverage for each test case separately (Section III-C). This process identified 5 741 flaky tests in 476 projects. 4 633 of them were identified as order-dependent (OD), 874 as non-order-dependent (NOD), and 234 as infrastructure flaky. These numbers are very similar to the ones we reported in our original study, except that now we observed less infrastructure flakiness. We also found fewer flaky tests overall, however, this can be attributed to the fact that we were unable to execute the test suites of 134 of the originally 1 006 projects. Despite using the same git revisions of the projects as the original study, this can happen, for example, due to third party libraries becoming incompatible to projects that did not specify the versions of their dependencies (in which case `pip` installs their latest version).

1) Sampling of flaky tests: Our evaluation is based on manually identified fault locations, which are not provided in the original dataset, but labeled by us as part of this study (Section IV-A2). Since this is an effortful process, we needed to take a sample of the flaky tests we found. We selected 89 NOD flaky tests from 51 projects using random stratified sampling, drawing a maximum of ten flaky tests per project to limit the influence of single projects with many flaky tests. In the same fashion, we selected 29 OD flaky tests from 5 projects. We put greater focus on NOD flakiness since SFFL is less qualified for locating OD faults (Section III-B4). In total, we sampled 118 flaky tests from 56 projects. Figure 1 shows statistics about these projects: The average (median) project contains 2 flaky and 17 stable tests and has 758 lines of source code. Note that despite applying random stratified sampling, for the fault localization itself we always consider the entire test suite including all flaky and all stable tests.

2) Ground truth / Manual fault localization: To build a ground truth that we can compare our approach against, we performed a manual fault localization: Using the SFFL results, the error messages, as well as the source code of the respective projects, four software engineering researchers labeled the fault locations of the flaky tests in our sample. Each flaky test was inspected individually by two researchers. For cases in which they disagreed, discussions were held until the disagreements were resolved. According to Fleiss' kappa [41], we reached an inter-rater reliability of 0.66, which is considered 'very good' according to Regier et al. [42]. Multiple fault locations could be assigned to a single flaky test. On average, the researchers assigned 1.19 fault locations per flaky test.

For 17 of the 118 sampled tests, the prerequisites for applying SFFL (Section III-B), and indeed any form of SFL, were not satisfied, so we had to exclude them. We discuss these cases in RQ1. This leaves us with 101 valid evaluation subjects (flaky tests) from 48 projects, of which 81 are non-order-dependent.

Apart from the locations of the flaky faults, we also manually determined the root causes of the flaky tests along established categories [13]–[15]. Similar to the study from which we sampled our projects [15], we found networking and randomness to be by far the most prevalent causes of NOD flakiness. The root cause labeling also identified one test that was categorized as order-dependent (only behaved flaky when run in random order, but not in same order), that was actually non-order-dependent flaky, as which we also consider it in the following analyses. We provide this data openly [20].

3) Metrics: To determine the performance of spectrum-based flaky fault localization, we compute the suspiciousness rank of each true fault. For a given statement s, its suspiciousness rank $rank(s)$ equals its position in the ranked list of statements, sorted by suspiciousness score in descending order. In other words, the suspiciousness rank is the number of statements one has to go through until finding the true fault. The best rank is 1, the worst rank equals the number of lines of code in the program. Since multiple statements can have the same suspiciousness score, ties can occur. To accommodate for such cases, we calculate three different ranks: the best-case rank (assuming the statement was listed as the first among its tied neighbors), the worst-case rank (assuming the statement was listed last within the tied group), and the average-case rank as the mean of the best-case and worst-case. If multiple true fault locations exist for one test case, the rank of the fault equals the minimum of the ranks of the individual locations, which is the number of statements that need to be processed until the first true fault location is found.

Comparing the suspiciousness ranks between different projects comes with the issue that larger projects (in terms of lines of source code) can skew results, simply because the mere chance of guessing the right statement is smaller for a bigger set of possible statements. To account for the projects' sizes, we also compute the EXAM score [43], which is the number of source code lines that need to be investigated until a first fault location is found relative to the total number of lines of source code. In other words, $EXAM(s) = \frac{rank(s)}{N}$, where N is the number of source code lines contained in the respective project. The value range of the EXAM score is $]0, 1]$. The best EXAM score is $1/N$, the worst EXAM score is 1.

4) RQ1 (Effectiveness): To assess how well SFFL can locate NOD flaky faults, we apply it to the 81 NOD flaky tests (from 45 projects) in our dataset, and compute suspiciousness scores using five established formulas also used in prior empirical studies on spectrum-based fault localization [23]: Tarantula [25], Ochiai [26], DStar [24], Op2 [27], and Barinel [28]. Based on these scores and the manually labeled fault locations, we calculate both the rank and the resulting EXAM scores of each fault. For counting the projects' lines of source code we used CLOC [44]. We compare the outcomes both among each other, determining the best performing formula, and against the performance of traditional SFL on deterministic faults [23].

5) RQ2 (Improvement): To measure the effects of SFFL's intersection-union-coverage-processing (Section III-A), we compare it against three baselines: The *single* baseline performs SFL using only the coverage obtained from the first test execution. It simulates the circumstances under which Habchi et al. [32] operate, where flaky tests are known, for example through a prediction, however, no historic coverage data is available, so a single execution is used to obtain it. The *union* baseline performs SFL considering a statement as covered by a test, if it was involved in any of its executions (which SFFL only does for stable tests). It thus considers all available execution data, simulating a naive approach at utilizing multiple coverage behaviors. The *individual* baseline considers multiple coverage behaviors of the same test case not by aggregating them into one coverage vector, but by treating each test execution like a separate flaky/stable test. $totalflaky$ and $totalstable$ therefore become the number of test executions of flaky and stable tests. We compare the EXAM scores reached by SFFL against each baseline and use a Wilcoxon signed-rank test [45]—which is a commonly used, non-parametric paired difference test—to check for statistically relevant differences.

6) RQ3 (Root Cause Differences): One of the main properties of a flaky test is its root cause, i.e., the broad category of its non-deterministic source, e.g., *networking* or *randomness*. To investigate if the root cause impacts the effectiveness of SFFL, we compute the EXAM score for each subset of tests sharing the same root cause. We identify statistically significant differences by using a Mann-Whitney U test [46] (an unpaired alternative to Wilcoxon) in combination with the Vargha and Delaney $\hat{A}_{12}$ effect size [47]. Although we specifically target NOD flaky tests with SFFL, we also include the 20 OD flaky tests for this research question to validate this design decision.

B. Threats to Validity

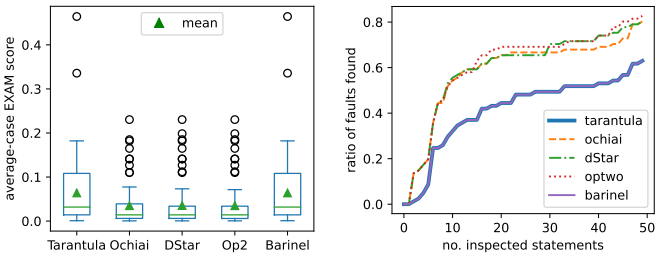
1) External validity: The flaky tests we used for our evaluation are taken from an existing dataset of flaky Python tests [15], which was also used by other researchers to evaluate flakiness detection and debugging techniques [18], [19], [48]. To avoid creating a bias towards projects containing more or less flaky tests, we applied random stratified sampling to derive a tangible subset for our manual fault localization. Nevertheless, we inherit any potentially existing bias in the original dataset.

2) Construct validity: We executed each test 800 times, which is twice as often as the original study that created the dataset. Nevertheless, some flaky tests have very low failure rates, which might therefore have been misclassified as non-flaky or OD flaky, while being NOD flaky. Concerning the metrics we use, there are known concerns about whether the EXAM score properly resembles the usefulness of a fault localization technique to a programmer [49]. However, SFFL is not only targeting manual debugging, but also means to serve as a foundation for other techniques, such as automated repair.

3) Internal validity: Providing the researchers with the SFFL results might influence them during the manual fault localization. On the other hand, the goal of our evaluation is to verify if SFFL is able to identify locations in the code that are helpful when debugging flakiness and locations where a possible fix *could* be applied. To judge in which of these locations the fix *should* be applied is only possible with an exact specification, which is not present for most projects.

Table IV: Evaluation results

	Worst-case		Average-case		Best-case	
	Mean	Median	Mean	Median	Mean	Median
Tarantula	0.085	0.038	0.064	0.032	0.043	0.018
Ochiai	0.054	0.023	0.035	0.014	0.016	0.005
DStar	0.053	0.023	0.035	0.014	0.017	0.005
Op2	0.053	0.023	0.035	0.014	0.017	0.005
Barinel	0.085	0.038	0.064	0.032	0.043	0.018



(a) average-case EXAM scores (b) rank progression

Figure 2: Effectiveness of SFFL on 81 NOD flaky tests

C. RQ1: Locating NOD Flaky Faults – Effectiveness

Table IV and Fig. 2a summarize the EXAM scores achieved by SFFL: Ochiai, DStar, and Op2 yielded the best and also very similar results, whereas Tarantula and Barinel delivered a notably worse performance. In the following discussions we use DStar as a reference, since it also excelled in other studies [21]. In terms of absolute EXAM scores, the effectiveness of SFFL is roughly in the same region as the one reached by SFL on deterministic faults [23], where DStar performed best with a mean EXAM score of 0.0404 (SFFL reached 0.0353), followed by Ochiai with 0.0405 (SFFL reached 0.0349). The fact that the median scores are always better than the mean scores is caused by poor performing outliers, which are depicted as circles (Fig. 2a). Figure 2b shows the progression of faults found after inspecting x statements. Using DStar, the faults of 55.6 % of all flaky tests were listed within the top ten most

```
768 def test_wrap_strategy(self):
769   cell = self.worksheet.get_values('A1', 'A1', returnas="range")[0][0]
770   assert cell.wrap_strategy == "WRAP_STRATEGY_UNSPECIFIED"
771   cell.wrap_strategy = "WRAP"
772   cell = self.worksheet.get_values('A1', 'A1', returnas="range")[0][0]
773   assert cell.wrap_strategy == "WRAP"
```

(a) test_wrap_strategy (`tests/online_test.py`)

```
350 def _execute_requests(self, request):
351   """Execute a request to the Google Sheets API v4.
352
353   When the API returns a 429 Error will sleep for the specified time and
                  try again.
354
355   :param request:     The request to be made.
356   :return:            Response
357   """
358   try:
359     response = request.execute(num_retries=self.retries)
360   except HttpError as error:
361     if error.resp['status'] == '429' and self.check:
362       time.sleep(self.seconds_per_quota)  # TODO use asyncio
363       response = request.execute(num_retries=self.retries)
364     else:
365       raise
366   return response
```

(b) Flaky fault (`pygsheets/sheet.py:359`)

Figure 3: Project `pygsheets`

suspicious statements. This number underlines the capabilities of SFFL in debugging flaky tests.

The test for which SFFL yielded the best results (average-case DStar EXAM score $0.000\,52$, rank 2) is `test_wrap_strategy` (Fig. 3a) from project `pygsheets`[1], a Google spreadsheets Python API. The test case intermittently fails with an HTTP 404 (not found) error. Given this error message, it is rather obvious that the flakiness is caused by network issues. However, the test itself does not contain any network interactions, leaving us with the question of where in the system under test the request is made. SFFL marked three statements (tied) with the highest DStar suspiciousness score for this project: lines 358 and 359 in `pygsheets/sheet.py` and line 48 in `pygsheets/spreadsheet.py`. Indeed the second of these locations is the request causing the HTTP error (Fig. 3b). One explanation for the good performance of SFFL on project `pygsheets` could be its number of tests: With 55 flaky and 5 stable tests, it possesses a comparably large test suite and the highest number of flaky test in our evaluation sample. This gives the fault localization much coverage data to build on, leading to fewer ties in the suspiciousness scores.

To illustrate the limitations of SFFL we consider the 17 tests that we excluded from our quantitative evaluation because pytest-cov [36] was unable to produce sufficient data, preventing the application of fault localization. On closer inspection, we found that most of these cases can be attributed to non-deterministic parametrizations that take place before the test function is entered and are therefore not reported as covered by the test. Specifically, 8 of these cases are caused by module-level fixtures and 5 are caused by Hypothesis [50], a property-based testing library for Python that dynamically parametrizes a test. One such case is `test_sort_property` (Fig. 4) from

[1]https://github.com/nithinmurali/pygsheets/tree/01b9548d

```
6  from hypothesis import given
7  from hypothesis.strategies import from_regex, integers, lists, tuples
8  # ... omitted ...
115 # Create filename "templates", ie before digits
116 strat_strings = from_regex(r"\A[^0-9]*\Z")
117 strat_mod = tuples(integers(), lists(integers(min_value=0), max_size=10))
118
119 strat = strat_strings, lists(strat_mod, max_size=5)
120
121
122 @given(lists(tuples(*strat)))
123 def test_sort_property(e):
124     strings = gen_test_data(e)
125     backup = strings[:]
126     random.shuffle(strings)
127     assert backup == sort(strings)
```

Figure 4: Flaky test in project `humansort` (`tests/test_sort.py`)

project `humansort`[2], a library that offers functionality to sort filenames and other strings in a human-readable way. The test case intermittently fails due to timeouts caused by Hypothesis. We marked line 122 as the fault location, since this is where the parametrization takes place. However, since this happens *before* the test function is entered, pytest-cov does not consider it to be covered by the test itself, which is why the statement is not part of the suspiciousness matrix. Since these issues stem from the coverage collection, this is not a limitation of SFFL specifically, but a general concern for any SFL approach.

Summary *(RQ1: Effectiveness)* Using the DStar formula, SFFL was able to achieve a mean average-case EXAM score of 0.035 (median 0.014) and locate the faults of $55.6\,\%$ of all flaky tests within ten statements.

D. RQ2: Locating NOD Flaky Faults – Improvement

Table V depicts the improvement yielded by SFFL over our three baselines (Section IV-A5). To quantify the overall improvement, we use the mean change in the EXAM score. We chose the mean over the median since latter is often zero for the *individual* baseline, despite SFFL yielding significant improvements. In fact, across all formulas and baselines, SFFL significantly improved the fault locating performance (p-value < 0.05). The superiority of SFFL can also be observed in Fig. 5, which shows the rank progression for all baselines: SFFL always performs at least as strong as the best baseline. Notably, the *union* baseline performed worse than the *single* baseline, which a Wilcoxon signed-rank test confirmed ($p < 0.001$). This shows that a naive usage of multiple coverage behaviors does not improve but may even diminish the effectiveness of SFL on flaky tests.

The fault on which SFFL exhibits the largest improvement ($72\,\%$ for DStar over the *single* baseline) was `test_sess_rx_interval_get` (Fig. 6a) from project `aiobfd`[3], a library implementing the BFD network protocol. The test case itself is rather short and its flakiness comes from

[2]https://github.com/coreygirard/humansort/tree/270c4ec6
[3]https://github.com/netedgeplus/aiobfd/tree/09012247

Table V: SFFL vs. baselines by average-case EXAM scores.

SFL formula	SFFL mean av.-case EXAM score	baseline = *single* SFFL better, unchanged, SFFL worse, not found	baseline = *single* mean impr.	baseline = *single* Wilcoxon p-value	baseline = *union* SFFL better, unchanged, SFFL worse, not found	baseline = *union* mean impr.	baseline = *union* Wilcoxon p-value	baseline = *individual* SFFL better, unchanged, SFFL worse, not found	baseline = *individual* mean impr.	baseline = *individual* Wilcoxon p-value
Tarantula	0.064	59, 21, 1, 2	28.1 %	<0.001	64, 16, 1, 2	40.4 %	<0.001	62, 18, 1, 2	39.1 %	<0.001
Ochiai	0.035	60, 21, 0, 2	19.0 %	<0.001	65, 16, 0, 2	31.7 %	<0.001	24, 51, 6, 2	8.4 %	<0.001
DStar	0.035	60, 21, 0, 2	18.7 %	<0.001	65, 16, 0, 2	31.5 %	<0.001	24, 51, 6, 2	8.0 %	<0.001
Op2	0.035	57, 24, 0, 2	17.0 %	<0.001	64, 17, 0, 2	30.0 %	<0.001	13, 62, 6, 2	2.5 %	<0.05
Barinel	0.064	59, 21, 1, 2	28.1 %	<0.001	64, 16, 1, 2	40.4 %	<0.001	62, 18, 1, 2	39.1 %	<0.001

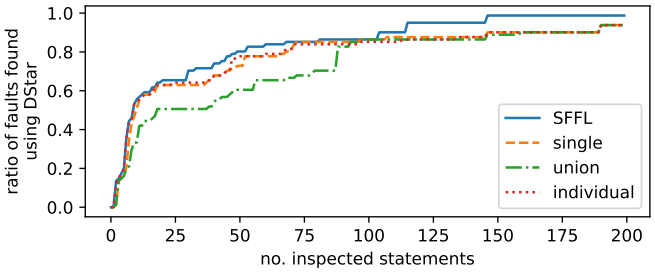


Figure 5: Rank progression of DStar compared to baselines

```
39 @pytest.fixture()
40 def session():
41     """Create a basic aiobfd session"""
42     return aiobfd.session.Session('127.0.0.1', '127.0.0.1')
       # ... omitted ...

217 def test_sess_rx_interval_get(session):
218     """Attempt to get the Required Min Rx Interval"""
219     assert session.required_min_rx_interval == 1000000
```

(a) test_sess_rx_interval_get (`tests/test_session.py`)

```
49 class Session:
50   """BFD session with a remote"""
51
52   def __init__(self, local, remote, family=socket.AF_UNSPEC, passive=
                  False,
53     tx_interval=1000000, rx_interval=1000000, detect_mult=3):
       # ... omitted ...

91     # Create the local client and run it once to grab a port
92     log.debug('Setting up UDP client for %s:%s.', remote, CONTROL_PORT)
93     src_port = random.randint(SOURCE_PORT_MIN, SOURCE_PORT_MAX)
94     fam, _, _, _, addr = socket.getaddrinfo(self.local, src_port)[0]
       # ... omitted ...
102    sock.bind(addr)
```

(b) Flaky fault (`aiobfd/session.py:93`)

Figure 6: Project `aiobfd`

its fixture[4], which creates a Session object using a random integer value for the port number (Fig. 6b, line 93). The test case fails intermittently in case the port is already in use, leading to an `OSError: [Errno 98] Address already in use` in line 102. In terms of root cause, this test case shows a mixture of local networking and randomness and we consider line 93 of the session module (Fig. 6b) as the faulty statement.

Table VIa depicts the corresponding ranked fault localization matrix using the *single* method. The two statements receiving

[4]native pytest fixtures are not affected by the coverage-collection issue that `humansort` faced (Fig. 4). They are reported as covered by the respective test.

Table VI: Suspiciousness matrices for project `aiobfd`

(a) using traditional SFL

rank	file (* = contains fault)	line	DStar susp. score
1,2	`tests/test_session.py`	219,353	1.0
3-47	`aiobfd/session.py`	148∼461	0.0727
48-95	`*aiobfd/session.py` `tests/test_session.py`	55-112,123 42	0.0714
96,97	`aiobfd/transport.py`	12,16	0.0702

(b) using SFFL

rank	file (* = contains fault)	line	DStar susp. score
1-39	`*aiobfd/session.py` `tests/test_session.py`	55-102 42	0.0714

the highest suspiciousness scores are the test code itself (line 219), as well as the code of the one other flaky test in the project (line 353). The following block (rank 3–47) contains multiple methods of the Session class. Then follows the constructor of the Session class (lines 55–112)—which contains the fault—as well as another method (line 123), and the session fixture (line 42). The first block (rank 3–47) received a slightly higher suspiciousness score, because it was covered by fewer non-flaky tests than the constructor. The faulty statement therefore receives an average-case suspiciousness rank of 71.5 (best-case 48, worst-case 95). Table VIb shows the ranked fault localization matrix using SFFL. By only considering statements that were covered by both passing and failing executions of flaky tests, the approach was able to cut down the number of possibly faulty statements from 97 to 39, crossing out the test code itself, the part of the constructor that is executed after the failure appeared (line 102), as well as other methods of the Session class. This project is a great example for how combining the coverage behavior of multiple test executions can increase the focus on actual faults.

In two special cases, SFFL was not able to locate the fault at all, although the baselines could, which was caused by its coverage intersection processing. In particular, the projects of these two cases are called `mpesa-py`[5] and `pympesa`, and both flaky tests are named `test_get_balance`. After manual inspection we found that they are probably undeclared forks. According to their README, they implement a wrapper

[5]https://github.com/Arlus/mpesa-py/tree/2844ac9c

```
1  import requests
2  from .auth import MpesaBase
3
4
5  class Balance(MpesaBase):
6    def __init__(self, env="sandbox", app_key=None, app_secret=None,
         sandbox_url="https://sandbox.safaricom.co.ke",
7                live_url="https://safaricom.co.ke"):
8      MpesaBase.__init__(self, env, app_key, app_secret, sandbox_url,
           live_url)
9      self.authentication_token = self.authenticate()
10
11   def get_balance(self, initiator=None, security_credential=None,
         command_id=None, party_a=None, identifier_type=None,
12     remarks=None, queue_timeout_url=None,result_url=None):
       # ... omitted ...
50     r = requests.post(saf_url, headers=headers, json=payload)
51     return r.json()
```

Figure 7: Project `mpesa-py` (`mpesa/api/balance.py`)

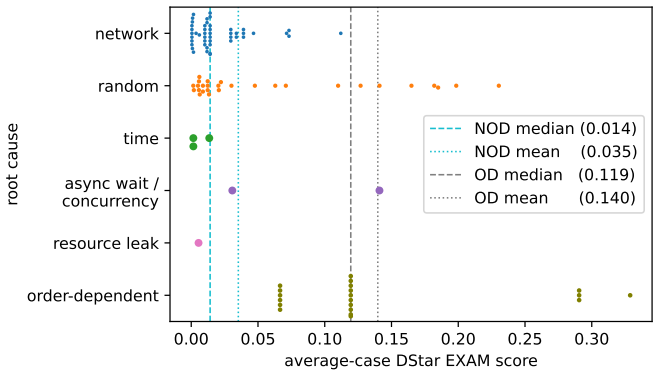


Figure 8: Swarm plot of EXAM scores grouped by root cause

providing convenient access to the API of MPESA, a mobile phone-based money transfer service.

The test creates a Balance object (Fig. 7) and calls the `get_balance` method, which issues a remote API request (line 50). Intermittent failures occur in case the web service is not available. However, network issues can also occur earlier during the authentication (line 9), which is in fact the cause for six other flaky tests in the projects. Since executions that experienced authentication failures did not reach line 50, SFFL did not consider the line to be consistently covered by the flaky test. As no other test covered it either, line 50 was excluded from the suspiciousness matrix and the fault was not found by SFFL. Consequently, this is a multi-fault localization problem, which is a well known limitation of SFL [28], but the effect was amplified by SFFL's coverage processing.

Summary ***(RQ2: Improvement)*** SFFL significantly improves fault localization on flaky tests. For DStar, the EXAM score improved in 60 out of 81 cases and by a mean of 18.7 % compared using a single test execution. When facing multiple faults, SFFL can however fail to locate all of them.

E. RQ3: Root Cause Differences

Table VII and Fig. 8 show the distribution of tests across root cause categories as well as the performance reached by SFFL on the respective subsets. While we found only small differences among NOD root causes, we see a strong gap between NOD and OD flaky tests: Latter differ significantly (p-value of Mann–Whitney U test < 0.001) with a large effect size ($\hat{A}_{12} = 0.90$). This confirms our intuition that SFFL is not well suited for OD tests, since it relies on many failing/flaky tests covering faulty statements, which is usually not the case for faults that are located inside a test case. For the same reason, Habchi et al. [32] excluded OD flaky tests from their evaluation of SFL on flaky tests. While order-dependency is a frequent cause of flakiness, we regard SFFL's confined ability to debug OD flaky tests as a negligible limitation, since specialized approaches for debugging order-dependencies exist [16]–[19]. NOD flaky tests, on the other, have not seen the same attention in terms of debugging approaches, which SFFL addresses.

For NOD flaky tests, we find significant differences only between network- and randomness-related flaky tests (p = 0.048). The effect size is small ($\hat{A}_{12} = 0.64$), and the difference is most likely caused by projects of the former simply containing more flaky tests on average (median 4) than projects with randomness-related flakiness (median 1), which helps SFFL to locate faults. Therefore, developers should be able to expect a similar performance from SFFL on NOD flaky tests, regardless of their exact root cause, which does also not need to be known to apply SFFL.

Table VII: Performance differences between root causes

root cause	no. tests	average-case DStar EXAM score		
		median	mean	std
network	47	0.014	0.020	0.023
random	28	0.021	0.061	0.073
order-dependent	20	0.119	0.140	0.086
time	3	0.002	0.006	0.007
async wait / concurrency	2	0.086	0.086	0.078
resource leak	1	0.005	0.005	-

Summary ***(RQ3: Root Cause Differences)*** SFFL performs expectedly worse on order-dependent flaky tests compared to non-order-dependent ones. Among different non-order-dependent root causes we found no substantial differences.

V. Related Work

A. The Impact of Flakiness on SFL

Several prior studies investigated the effects of flaky tests on fault localization: Cordy et al. [29] investigated the impact of flakiness on automated program repair (of which fault localization is the first step) using Defects4J [51] and found that the fault localization step is particularly sensitive to test flakiness: Even the smallest degree of flakiness was sufficient to diminish both precision and recall of threshold-based suspicious statement selection by more than 80 %. This can have disastrous effects on patch generation, which was independently confirmed by another study on automatic repair [52]. Qin et al. [30] also studied the impact of flaky tests on automated program repair based on tests that have been annotated as flaky in the Defects4J defect benchmark [51], and also found flakiness to significantly decrease fault localization efficacy, with localization results decreasing for 90 out of 387 bugs and none improving after including flaky tests. Vancsics et al. [31] simulated the effect of

flakiness on fault localization using 28 bugs from the Defects4J dataset [51]. They found that after adding flakiness (without changing coverage), most bugs received a suspiciousness rank that was within $\pm 25\%$ of the original. All of these studies motivate the need to investigate SFL in the face of flakiness, which we do in this paper by proposing SFFL. Since SFFL produces consistent results despite varying test outcomes, it can absorb the effect of flakiness and form a solid foundation for automated program repair and other techniques building on fault localization.

B. Flakiness Debugging

Habchi et al. [32] applied traditional SFL to flaky tests, aiming to pinpoint the classes that cause the non-deterministic behavior of flaky tests. Apart from using the tests' coverage behavior in the fashion of our *single* baseline (RQ2), they also utilize the change history and static code metrics. Their results indicate that this augmentation improves fault localization performance, ranking $47\,\%$ of faulty classes in the top three most suspicious components, compared to $39\,\%$ without. Interestingly, the DStar formula yielded the worst performance, whereas this formula performed best in our experiments. Unlike us, they do not consider the coverage behavior of both passing and failing executions of flaky tests (due to difficulties in reproducing them), which they also mention as a possible threat to validity. Furthermore, SFFL works on the statement level instead of the class level and focuses only on coverage behavior, not other data sources such as change history and static code metrics. The latter might, however, be a path worthwhile pursuing and a possible direction for future work.

Moran et al. proposed FlakyLoc [33], a spectrum-based technique to identify environment factors leading to test flakiness. They evaluated FlakyLoc on one flaky test where it was able to correctly identify that its intermittent failures were caused by a low screen resolution. Unlike our approach, they deliberately alter the spectrum-based components (in their case environment factors) in a combinatorial fashion to identify the one responsible for the failure. Coverage-based SFL (and thus SFFL) is not able to adopt this approach, since the spectrum-based components are code elements.

Ziftci and Cavalcanti [34] proposed the Flakiness Debugger. Like SFFL, it processes multiple coverage behaviors of flaky tests to determine the location of the flaky fault. Unlike SFFL, it is not based on SFL, but on the custom `DIVERGENCE` algorithm that highlights the first divergence in the control flow between passing and failing executions. Therefore, it also requires a more complex instrumentation, which does not only collect code coverage (which lines were covered), but full execution traces (the order in which the lines were executed). Such instrumentation is tool-wise less supported and can have considerable impact on program behavior and therefore test flakiness. They evaluated their approach on 83 fixed flaky tests, for which it was able to point to relevant code locations involved in flakiness with $81.93\,\%$ accuracy. In our evaluation, SFFL was able to identify the faults of $55.6\,\%$ of flaky tests within the top ten most suspicious statements. While being similar on a technical level, the Flakiness Debugger (and its evaluation) is targeted more towards aiding the manual debugging process, which is also why they preferred the `DIVERGENCE` algorithm over SFL. SFFL can also be used for manual debugging, however, it can also build the foundation for automated repair of flaky tests, which Ziftci and Cavalcanti [34] found to be in high demand by developers.

Ahmad et al. [48] introduced FLAKYPY, a tool that builds on the `DIVERGENCE` algorithm and aims to improve the localization of randomness-related flaky tests by taking the values of local variables and anomalies between them in different runs into account. Applied to 26 projects taken from the same dataset as we use [15], FLAKYPY was unable to reproduce the flaky behavior for 22 of them. In 5 cases FLAKYPY was not able to find a potential root cause. Unlike FLAKYPY, SFFL is not root cause specific, apart from targeting NOD flakiness.

VI. Conclusions

In this paper we introduced SFFL (Spectrum-based Flaky Fault Localization), an extension of SFL that specifically targets the debugging of flaky tests. SFFL considers the fact that a test can have multiple coverage behaviors, and harnesses this property by computing the coverage intersection for flaky tests and the union for stable ones. By doing so, SFFL remains mostly deterministic while achieving a substantial improvement over traditional SFL: In our evaluation conducted on 81 non-order-dependent flaky tests it improved the average-case EXAM score in 60 cases and by a mean of $18.7\,\%$. In absolute terms, SFFL reached an average-case EXAM score of 0.035 and the faults of $55.6\,\%$ of all flaky tests could be located within the top ten most suspicious statements.

A current limitation of our implementation of SFFL is that it is based on a single-fault assumption, and we found that multiple fault problems may even be amplified by SFFL's coverage processing. An important aspect of future work will thus be to investigate extending SFFL considering existing work on multi-fault SFL. While there were no substantial differences between flaky tests of different non-order-dependent root causes, a general limitation of SFFL is that it targets non-order-dependent tests. Consequently, future work could investigate generalizing SFFL to order-dependent flaky tests, where the root cause for flakiness lies in the test code rather than the code under test. Furthermore, it could try to apply SFFL in different contexts, such as other programming languages.

We hope that SFFL will both serve developers, making debugging flaky tests more convenient and efficient, and that it will inspire and be of use to future research, for example as a foundation for automated fixing or more sophisticated debugging techniques. We make all data openly available [20].

Acknowledgements

We thank Christian Kasberger for his contributions to this paper in form of an early pre-study during his bachelor thesis. We furthermore thank Florian Kroiss, Philipp Fink, and Ammar Harrat for manually labeling fault locations.

References

[1] M. Gruber and G. Fraser, "A survey on how test flakiness affects developers and what support they need to address it," in *International Conference on Software Testing, Verification and Validation (ICST)*, 2022, pp. 82–92.

[2] O. Parry, G. M. Kapfhammer, M. Hilton, and P. McMinn, "Surveying the developer experience of flaky tests," in *International Conference on Software Engineering: Software Engineering in Practice Track (ICSE-SEIP)*, 2022, pp. 253–262.

[3] J. Bell, O. Legunsen, M. Hilton, L. Eloussi, T. Yung, and D. Marinov, "DeFlaker: Automatically detecting flaky tests," in *International Conference on Software Engineering (ICSE)*, 2018, pp. 433–444.

[4] W. Lam, R. Oei, A. Shi, D. Marinov, and T. Xie, "iDFlakies: A framework for detecting and partially classifying flaky tests," in *International Conference on Software Testing, Verification and Validation (ICST)*, 2019, pp. 312–322.

[5] D. Silva, L. Teixeira, and M. d'Amorim, "Shake it! Detecting flaky tests caused by concurrency with Shaker," in *International Conference on Software Maintenance and Evolution (ICSME)*, 2020, pp. 301–311.

[6] G. Pinto, B. Miranda, S. Dissanayake, M. d'Amorim, C. Treude, and A. Bertolino, "What is the vocabulary of flaky tests?" in *International Conference on Mining Software Repositories (MSR)*, 2020, pp. 492–502.

[7] A. Alshammari, C. Morris, M. Hilton, and J. Bell, "FlakeFlagger: Predicting flakiness without rerunning tests," in *International Conference on Software Engineering (ICSE)*, 2021, pp. 1572–1584.

[8] R. Verdecchia, E. Cruciani, B. Miranda, and A. Bertolino, "Know you neighbor: Fast static prediction of test flakiness," *IEEE Access*, pp. 76 119–76 134, 2021.

[9] S. Fatima, T. A. Ghaleb, and L. Briand, "Flakify: A black-box, language model-based predictor for flaky tests," *IEEE Transactions on Software Engineering*, pp. 1–17, 2022.

[10] Y. Qin, S. Wang, K. Liu, B. Lin, H. Wu, L. Li, X. Mao, and T. F. D. A. Bissyande, "Peeler: Learning to effectively predict flakiness without running tests," in *International Conference on Software Maintenance and Evolution (ICSME)*, 2022, pp. 1–12.

[11] O. Parry, G. M. Kapfhammer, M. Hilton, and P. McMinn, "Evaluating features for machine learning detection of order-and non-order-dependent flaky tests," in *International Conference on Software Testing, Verification and Validation (ICST)*, 2022, pp. 93–104.

[12] C. Li and A. Shi, "Evolution-aware detection of order-dependent flaky tests," in *International Symposium on Software Testing and Analysis (ISSTA)*, 2022, p. 114–125.

[13] Q. Luo, F. Hariri, L. Eloussi, and D. Marinov, "An empirical analysis of flaky tests," in *International Symposium on Foundations of Software Engineering (FSE)*, 2014, pp. 643–653.

[14] M. Eck, F. Palomba, M. Castelluccio, and A. Bacchelli, "Understanding flaky tests: The developer's perspective," in *Joint Meeting of the European Software Engineering Conference and the Symposium on the Foundations of Software Engineering (ESEC/FSE)*, 2019, pp. 830–840.

[15] M. Gruber, S. Lukasczyk, F. Kroiß, and G. Fraser, "An empirical study of flaky tests in Python," in *International Conference on Software Testing, Verification and Validation (ICST)*, 2021, pp. 148–158.

[16] A. Gambi, J. Bell, and A. Zeller, "Practical test dependency detection," in *International Conference on Software Testing, Verification and Validation (ICST)*, 2018, pp. 1–11.

[17] A. Shi, W. Lam, R. Oei, T. Xie, and D. Marinov, "iFixFlakies: A framework for automatically fixing order-dependent flaky tests," in *Joint Meeting of the European Software Engineering Conference and the Symposium on the Foundations of Software Engineering (ESEC/FSE)*, 2019, pp. 545–555.

[18] A. Wei, P. Yi, Z. Li, T. Xie, D. Marinov, and W. Lam, "Preempting flaky tests via non-idempotent-outcome tests," in *International Conference on Software Engineering (ICSE)*, 2022, pp. 1730–1742.

[19] W. L. Ruixin Wang, Yang Chen, "iPFlakies: A framework for detecting and fixing python order-dependent flaky tests," in *International Conference on Software Engineering: Companion Proceedings (ICSE Companion)*, 2022, pp. 120–124.

[20] M. Gruber and G. Fraser, "Debugging flaky tests using spectrum-based fault localization [dataset]," 2023. [Online]. Available: https://doi.org/10.6084/m9.figshare.21901401

[21] W. E. Wong, R. Gao, Y. Li, R. Abreu, and F. Wotawa, "A survey on software fault localization," *IEEE Transactions on Software Engineering*, vol. 42, no. 8, pp. 707–740, 2016.

[22] L. Gazzola, D. Micucci, and L. Mariani, "Automatic software repair: a survey," in *International Conference on Software Engineering (ICSE)*, 2018, p. 1219.

[23] S. Pearson, J. Campos, R. Just, G. Fraser, R. Abreu, M. D. Ernst, D. Pang, and B. Keller, "Evaluating and improving fault localization," in *International Conference on Software Engineering (ICSE)*, 2017, pp. 609–620.

[24] W. E. Wong, V. Debroy, R. Gao, and Y. Li, "The DStar method for effective software fault localization," *IEEE Transactions on Reliability*, vol. 63, no. 1, pp. 290–308, 2014.

[25] J. A. Jones, M. J. Harrold, and J. Stasko, "Visualization of test information to assist fault localization," in *International Conference on Software Engineering (ICSE)*, 2002, pp. 467–477.

[26] R. Abreu, P. Zoeteweij, R. Golsteijn, and A. J. C. van Gemund, "A practical evaluation of spectrum-based fault localization," *Journal of Systems and Software*, vol. 82, no. 11, pp. 1780–1792, 2009.

[27] L. Naish, H. J. Lee, and K. Ramamohanarao, "A model for spectra-based software diagnosis," *ACM Transactions on Software Engineering and Methodology*, pp. 11:1–11:32, 2011.

[28] R. Abreu, P. Zoeteweij, and A. J. C. van Gemund, "Spectrum-based multiple fault localization," in *International Conference on Automated Software Engineering (ASE)*, 2009, pp. 88–99.

[29] M. Cordy, R. Rwemalika, A. Franci, M. Papadakis, and M. Harman, "FlakiMe: Laboratory-controlled test flakiness impact assessment," in *International Conference on Software Engineering (ICSE)*, 2022, pp. 982–994.

[30] Y. Qin, S. Wang, K. Liu, X. Mao, and T. F. Bissyandé, "On the impact of flaky tests in automated program repair," *International Conference on Software Analysis, Evolution and Reengineering, SANER*, pp. 295–306, 2021.

[31] B. Vancsics, T. Gergely, and Á. Beszédes, "Simulating the effect of test flakiness on fault localization effectiveness," in *Workshop on Validation, Analysis and Evolution of Software Tests (VST)*, 2020, pp. 28–35.

[32] S. Habchi, G. Haben, J. Sohn, A. Franci, M. Papadakis, M. Cordy, and Y. L. Traon, "What made this test flake? pinpointing classes responsible for test flakiness," in *International Conference on Software Maintenance and Evolution (ICSME)*, 2022, pp. 352–363.

[33] J. Morán Barbón, C. Augusto Alonso, A. Bertolino, C. A. Riva Álvarez, P. J. Tuya González *et al.*, "Flakyloc: flakiness localization for reliable test suites in web applications," *Journal of Web Engineering, 2*, 2020.

[34] D. C. Celal Ziftci, "De-Flake your tests: Automatically locating root causes of flaky tests in code at Google," in *International Conference on Software Maintenance and Evolution (ICSME)*, 2020, pp. 736–745.

[35] M. R. Hoffmann, B. Janiczak, and E. Mandrikov, "Eclemma-jacoco java code coverage library," 2011. [Online]. Available: https://www.eclemma.org/jacoco/

[36] "pytest-cov: a pytest plugin that produces coverage reports." [Online]. Available: https://pytest-cov.readthedocs.io/en/v2.8.1/

[37] J. Micco, "Flaky tests at Google and how we mitigate them," 2016. [Online]. Available: https://testing.googleblog.com/2016/05/flaky-tests-at-google-and-how-we.html

[38] S. Elbaum, G. Rothermel, and J. Penix, "Techniques for improving regression testing in continuous integration development environments," in *International Symposium on Foundations of Software Engineering (FSE)*, 2014, pp. 235–245.

[39] "pytest." [Online]. Available: https://docs.pytest.org/en/7.2.x/

[40] "pytest-cov contexts." [Online]. Available: https://pytest-cov.readthedocs.io/en/v2.8.1/contexts.html

[41] J. L. Fleiss, "Measuring nominal scale agreement among many raters." *Psychological bulletin*, p. 378, 1971.

[42] D. A. Regier, W. E. Narrow, D. E. Clarke, H. C. Kraemer, S. J. Kuramoto, E. A. Kuhl, and D. J. Kupfer, "Dsm-5 field trials in the united states and canada, part ii: test-retest reliability of selected categorical diagnoses," *American journal of psychiatry*, pp. 59–70, 2013.

[43] W. E. Wong, T. Wei, Y. Qi, and L. Zhao, "A crosstab-based statistical method for effective fault localization," in *International Conference on Software Testing, Verification and Validation (ICST)*, 2008, pp. 42–51.

[44] A. Danial, "cloc: v1.92," 2021. [Online]. Available: https://doi.org/10.5281/zenodo.5760077

[45] F. Wilcoxon, "Individual comparisons by ranking methods," *Biometrics Bulletin*, pp. 80–83, 1945.

[46] H. B. Mann and D. R. Whitney, "On a test of whether one of two random variables is stochastically larger than the other," *The annals of mathematical statistics*, pp. 50–60, 1947.

[47] A. Vargha and H. D. Delaney, "A critique and improvement of the cl common language effect size statistics of mcgraw and wong," *Journal of Educational and Behavioral Statistics*, pp. 101–132, 2000.

[48] A. Ahmad, E. N. Held, O. Leifler, and K. Sandahl, "Identifying randomness related flaky tests through divergence and execution tracing," in *International Conference on Software Testing, Verification and Validation (ICST)*, 2022, pp. 293–300.

[49] C. Parnin and A. Orso, "Are automated debugging techniques actually helping programmers?" in *International Symposium on Software Testing and Analysis (ISSTA)*, 2011, pp. 199–209.

[50] D. Maciver and Z. Hatfield-Dodds, "Hypothesis: A new approach to property-based testing," *J. Open Source Softw.*, vol. 4, no. 43, p. 1891, 2019.

[51] R. Just, D. Jalali, and M. D. Ernst, "Defects4j: A database of existing faults to enable controlled testing studies for java programs," in *International Symposium on Software Testing and Analysis (ISSTA)*, 2014, pp. 437–440.

[52] M. Martinez, T. Durieux, R. Sommerard, J. Xuan, and M. Monperrus, "Automatic repair of real bugs in java: A large-scale experiment on the defects4j dataset," *Empirical Software Engineering*, vol. 22, no. 4, pp. 1936–1964, 2017.

All online resources accessed on 2023–01–16.